House Price Prediction using Advanced Regression Techniques

Action Plan

1. Problem Statement

The objective of this project is to build a machine learning system capable of predicting residential house sale prices based on structural, locational, and condition-based features, using advanced regression techniques rather than a single baseline model. The project further aims to demonstrate a complete, production-oriented workflow — from raw data to a deployed, interactive prediction tool.

2. Dataset Selection

The Ames Housing dataset (Kaggle: 'House Prices - Advanced Regression Techniques') was selected over the more commonly used Boston Housing dataset. Ames offers a larger, richer feature set (79 explanatory variables vs. 13), and avoids the well-documented ethical issues associated with a racially-correlated proxy feature present in the Boston dataset, which led to its removal from scikit-learn.

3. Methodology Overview

- Exploratory Data Analysis — target distribution, skewness, correlation ranking, scatter plot review.
- Data Cleaning — selective outlier removal (justified by contradiction with expected price relationships, not blanket statistical thresholds), structural vs. genuine missing-value imputation.
- Feature Engineering — domain-driven features (TotalSF, TotalBath, HouseAge, YearsSinceRemodel), skew correction via log transform, one-hot encoding of categorical variables.
- Modeling — five required techniques (Linear Regression baseline, Ridge, Lasso, Random Forest, XGBoost) evaluated on a held-out validation split using RMSE, MAE, and R^2.
- Optimization — Bayesian hyperparameter tuning of XGBoost via Optuna (50 trials); stacked ensemble combining Ridge, Lasso, and tuned XGBoost via a Ridge meta-model.
- Explainability — SHAP value analysis to validate and interpret model behavior.
- Deployment — model persistence via joblib; interactive Streamlit web application for live predictions, including a build-year/renovation impact visualization.
- Engineering Practice — modular project structure (src/, notebooks/, app/, tests/), unit tests via pytest, version-controlled GitHub repository.

4. Tools and Technology Stack

- Language: Python 3.13
- Data handling: pandas, numpy
- Visualization: matplotlib, seaborn, plotly
- Modeling: scikit-learn (Ridge, Lasso, Random Forest), XGBoost
- Optimization: Optuna
- Explainability: SHAP
- Deployment: Streamlit

- Testing: pytest
- Version control: Git / GitHub
- Environment: VS Code, Python virtual environment (venv)

5. Timeline

Phase	Task	Days
1. Setup, Research & EDA	Environment setup, dataset selection (Ames Housing), literature review, missing value analysis, target distribution, correlation analysis, scatter plots	Day 1
2. Data Cleaning & Feature Engineering	Outlier removal, structural vs. genuine missing-value imputation, TotalSF/TotalBath/HouseAge/YearsSinceRemodel, one-hot encoding, log transforms	Day 2
3. Modeling, Tuning & Ensembling	Linear Regression, Ridge, Lasso, Random Forest, XGBoost baseline evaluation; Optuna hyperparameter tuning; stacked ensemble (Ridge + Lasso + XGBoost)	Day 3
4. Explainability & Deployment	SHAP summary and force plots; model persistence (joblib); Streamlit app build and styling	Day 4
5. Documentation & Submission	GitHub repo structure, README, unit tests, research report, action plan, final review and submission	Day 5

6. Challenges Encountered and Resolutions

Several practical issues arose during implementation, each resolved through standard debugging practice rather than starting over — a realistic reflection of applied data science work:

- Kernel/environment mismatch: an initially selected Jupyter kernel pointed to an empty '.venv' environment rather than the intended 'venv', causing ModuleNotFoundError on import. Resolved by explicitly re-registering and reselecting the correct kernel.
- Pandas type error during skewness calculation: a mixed-type column caused '.apply(skew)' to fail. Resolved by restricting the operation to select_dtypes(include=[np.number]) rather than relying on dtype != 'object'.
- Lasso convergence warning: the default max_iter was insufficient for the encoded feature space (269 columns). Resolved by feature scaling (StandardScaler) and increasing max_iter.
- Plotly color validation error: an 8-digit hex color with alpha channel (valid in CSS) was rejected by Plotly's color validator. Resolved by switching to an explicit rgba() string.

7. Expected Outcomes

The project is expected to deliver: a validated comparison of five regression techniques plus a stacked ensemble; a tuned, explainable final model; a working, deployed prediction interface; and a fully documented, version-controlled codebase suitable for both academic submission and public portfolio use.